

Understanding Query Optimization Bugs in Graph Database Systems

Yuyu Chen, Zhongxing Yu

Shandong University, Qingdao, China
ASPLOS '26, March 22–26, 2026

Core Problem & Existing Methods

Core Problem:

- Query optimization is critical for GDBMS performance but highly complex.
- Involves normalization, plan generation, cost estimation, and rule-based rewriting.
- Prone to correctness issues, analogous to compiler optimization bugs.

Core Problem & Existing Methods

Core Problem:

- Query optimization is critical for GDBMS performance but highly complex.
- Involves normalization, plan generation, cost estimation, and rule-based rewriting.
- Prone to correctness issues, analogous to compiler optimization bugs.

Drawbacks of Existing Testing Methods:

- **Broad Focus:** DT/MT tools (e.g., GDSmith, GraspDB) target all bug types, losing coverage on optimization-specific defects.
- **Lack of Guidance:** Rely on random generation or generic oracles, missing deep optimization logic.
- **Inefficient Oracles:** High cost for result validation; ignore cheap failure symptoms.

First Large-Scale Study:

- Analyzed **102 optimization bugs** across 4 GDBMSs (Neo4j, Memgraph, RedisGraph, Kuzu)
- Extracted patterns on *causes, manifestations, fixes*

Contributions & Methodology

First Large-Scale Study:

- Analyzed **102 optimization bugs** across 4 GDBMSs (Neo4j, Memgraph, RedisGraph, Kuzu)
- Extracted patterns on *causes, manifestations, fixes*

Key Findings (totaly 11):

- Plan space errors dominate (40/69 bugs); simple/empty LPGs often suffice
- **32%** of triggers use subqueries; **40%** feature multi-variable interactions

First Large-Scale Study:

- Analyzed **102 optimization bugs** across 4 GDBMSs (Neo4j, Memgraph, RedisGraph, Kuzu)
- Extracted patterns on *causes, manifestations, fixes*

Key Findings (totaly 11):

- Plan space errors dominate (40/69 bugs); simple/empty LPGs often suffice
- **32%** of triggers use subqueries; **40%** feature multi-variable interactions

Finding-Guided Tool:

- Generates simple LPGs + Cypher queries with subqueries/multi-variable interactions
- Uses internal errors, crashes, and wrong results as lightweight oracles
- Found **20 unique bugs** in Neo4j & Memgraph (10 are optimization bugs), all confirmed

Cypher Query Optimization Background

Query optimization transforms Cypher queries into efficient execution plans without changing the original syntax.

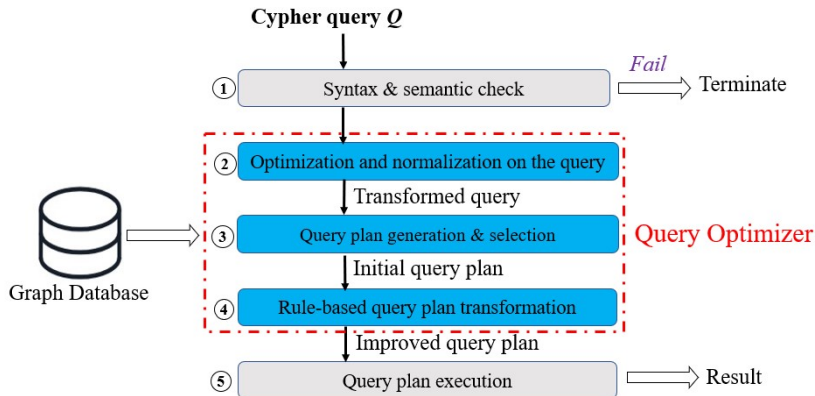


Figure: An overview of Cypher query execution steps

Cypher Query Optimization Examples 1 & 2

Example 1: Index Scan Replacement

Original Query

```
MATCH (p:Person) WHERE p.age > 30 RETURN p.name
```

Before Optimization:

ScanAll(p) → Filter(p.age>30) → Project

After Optimization:

ScanAllByIndex(p, age) → Project

Cypher Query Optimization Examples 1 & 2

Example 1: Index Scan Replacement

Original Query

```
MATCH (p:Person) WHERE p.age > 30 RETURN p.name
```

Before Optimization:

ScanAll(p) $\rightarrow$ Filter(p.age>30) $\rightarrow$ Project

After Optimization:

ScanAllByIndex(p, age) $\rightarrow$ Project

Example 2: Constant Folding & Alias Expansion

Original Query

```
MATCH (m:Movie) WHERE m.year = 1990+10 WITH m.title AS name RETURN name
```

- **Constant Folding:** $1990 + 10 \rightarrow 2000$ (precomputed at optimization time)
- **Alias Expansion:** Inline m.title to avoid redundant projection

Cypher Query Optimization Examples 3 & 4

Example 3: Subquery Un-nesting & Merging

Original Query

```
MATCH (a:Actor) CALL {MATCH (a)-[:ACTED]->(m:Movie) RETURN count(m)} RETURN a
```

- Merge subquery pattern with outer pattern → single graph traversal
- Replace row-by-row execution with HashAggregate

Cypher Query Optimization Examples 3 & 4

Example 3: Subquery Un-nesting & Merging

Original Query

```
MATCH (a:Actor) CALL {MATCH (a)-[:ACTED]->(m:Movie) RETURN count(m)} RETURN a
```

- Merge subquery pattern with outer pattern → single graph traversal
- Replace row-by-row execution with HashAggregate

Example 4: Graph State-Aware Predicate Pushdown

Original Query

```
MATCH (u:User)-[:FRIEND]->(v:User) WHERE v.status = 'active' WITH v WHERE  
v.lastLogin > datetime() - duration('P30D') RETURN v.name
```

- Push multiple WHERE conditions into index scan or expand operators
- Avoid generating redundant intermediate results

Methodology & Finding 1: Prevalence of Optimization Bugs

Methodology:

- Analyzed issue trackers of 4 Cypher-based GDBMSs: Neo4j, Memgraph, RedisGraph, Kuzu.
- Collected 102 representative optimization bugs via keyword search + manual validation.
- Examined issue reports, source code, patches, and test cases.

Methodology & Finding 1: Prevalence of Optimization Bugs

Methodology:

- Analyzed issue trackers of 4 Cypher-based GDBMSs: Neo4j, Memgraph, RedisGraph, Kuzu.
- Collected 102 representative optimization bugs via keyword search + manual validation.
- Examined issue reports, source code, patches, and test cases.

Finding 1:

Basic Content

Optimization bugs are prevalent across all four studied GDBMSs.

Source & Value

Source: Distribution analysis (Neo4j: 42, Memgraph: 11, RedisGraph: 29, Kuzu: 20).

Value: Confirms optimization as a systemic vulnerability; calls for dedicated research and specialized testing.

Bug Cause Finding 2: Optimization & Normalization Phase

Basic Content

Optimizers frequently overlook specific legal syntax formats and semantic attributes during normalization.

Bug Cause Finding 2: Optimization & Normalization Phase

Basic Content

Optimizers frequently overlook specific legal syntax formats and semantic attributes during normalization.

Source & Value

Source: 17 bugs analyzed; 10 ignore special formats, 3 lack semantic analysis.

Value: Highlights the critical transition from syntax/semantic check to optimization. Backend must perform thorough checks instead of blind assumptions.

Bug Cause Finding 2: Optimization & Normalization Phase

Basic Content

Optimizers frequently overlook specific legal syntax formats and semantic attributes during normalization.

Source & Value

Source: 17 bugs analyzed; 10 ignore special formats, 3 lack semantic analysis.

Value: Highlights the critical transition from syntax/semantic check to optimization. Backend must perform thorough checks instead of blind assumptions.

Specific Example: RedisGraph-634

Issue: Optimization precomputes function calls but fails to exclude dynamic zero-arg functions like `rand()`.

Fix: Explicitly forbid constant folding for zero-parameter dynamic functions.

Bug Cause Finding 3: Plan Generation & Selection Phase

Basic Content

A significant portion of bugs arises from incomplete or incorrect query plan space construction.

Bug Cause Finding 3: Plan Generation & Selection Phase

Basic Content

A significant portion of bugs arises from incomplete or incorrect query plan space construction.

Source & Value

Source: 69 bugs in this phase; 40 due to flawed plan space (graph state mismanagement, clause misinterpretation).

Value: Recommends formalizing Cypher clause semantics and dynamically maintaining graph state (structure, schema, context).

Bug Cause Finding 3: Plan Generation & Selection Phase

Basic Content

A significant portion of bugs arises from incomplete or incorrect query plan space construction.

Source & Value

Source: 69 bugs in this phase; 40 due to flawed plan space (graph state mismanagement, clause misinterpretation).

Value: Recommends formalizing Cypher clause semantics and dynamically maintaining graph state (structure, schema, context).

Specific Example: RedisGraph-1491

Issue: Property update logic ignores entity deletion status, causing crashes during plan generation.

Fix: Add precondition checks for entity deletion before attribute updates.

Bug Cause Findings 4 & 5: Cost, Exploration & Rule Transformation

Finding 4: Cost estimation and plan exploration algorithms are prone to implementation bugs.

- *Source:* 23 bugs (9 estimation, 14 exploration). Cardinality estimation and join ops are hotspots.
- *Value:* Mandates rigorous unit testing for core algorithms; prefer standard APIs over custom implementations.

Bug Cause Findings 4 & 5: Cost, Exploration & Rule Transformation

Finding 4: Cost estimation and plan exploration algorithms are prone to implementation bugs.

- *Source:* 23 bugs (9 estimation, 14 exploration). Cardinality estimation and join ops are hotspots.
- *Value:* Mandates rigorous unit testing for core algorithms; prefer standard APIs over custom implementations.

Finding 5: Rule-based plan transformation suffers from incomplete precondition checks and incorrect operator assumptions.

- *Source:* 9 bugs (6 incomplete checks, 3 wrong assumptions).
- *Value:* Suggests borrowing compiler verification tech (equality saturation, translation validation) for rule safety.

Bug Cause Findings 4 & 5: Cost, Exploration & Rule Transformation

Finding 4: Cost estimation and plan exploration algorithms are prone to implementation bugs.

- *Source:* 23 bugs (9 estimation, 14 exploration). Cardinality estimation and join ops are hotspots.
- *Value:* Mandates rigorous unit testing for core algorithms; prefer standard APIs over custom implementations.

Finding 5: Rule-based plan transformation suffers from incomplete precondition checks and incorrect operator assumptions.

- *Source:* 9 bugs (6 incomplete checks, 3 wrong assumptions).
- *Value:* Suggests borrowing compiler verification tech (equality saturation, translation validation) for rule safety.

Specific Example: Memgraph-1648

Issue: Assumes Filter always follows EdgeUniquenessFilter, breaking Cartesian $\rightarrow$ Join rewriting.

Fix: Remove hard-coded assumption and redesign transformation logic.

Bug Manifestation Findings 6 & 7: Input Attributes

Finding 6: Exposing optimization bugs rarely requires complex LPGs.

- *Source/Value:* 75% of bugs use simple graphs (<50 nodes/edges); 37% trigger on **empty graphs**. Enables lightweight, high-throughput testing.

Finding 7: Subqueries are high-risk triggers for optimization bugs.

- *Source/Value:* 32% of bug-exposing queries use subqueries (often nested). Existing tools lack support; future tools must natively generate nested subqueries.

Example of Finding 7

Specific Example: Neo4j-12760

Trigger: Nested CALL{} subqueries with UNION. Optimizer fails to preserve ORDER BY semantics across subquery boundaries in pipelined runtime.

Bug-Exposing LPG:

```
CREATE ({num:2})-[:r1]->({num:0})-[:r2]->({num:1})
```

Trigger Query:

```
MATCH (x) WITH x ORDER BY x.num
WITH x CALL {
  WITH x CALL {
    WITH x MATCH (x)-[:r1]->() RETURN y
    UNION
    WITH x MATCH (x)-[:r2]->() RETURN y
  }
  WITH y RETURN collect(y) as y
} WITH x RETURN x.num
```


Bug Manifestation Findings 8 & 9: Query Features & Failure Symptoms

Finding 8: 40% of queries feature *Single Clause Multiple Bound Variables Interaction*.

- *Source/Value*: Increases scope switching and data dependency complexity. Requires tools to model variable flow and prioritize reuse.

Finding 9: Failure symptoms are diverse but detectable cheaply.

- *Source*: Internal Error (24), Crash (24), Perf (23), Wrong Result (15).
- *Value*: Internal errors + crashes = 56%. Ideal for low-cost oracles without expensive result diffing.

Example of Finding 8

Specific Example: Neo4j-9435

Trigger: Variables k and t bound in first MATCH are reused/interacted in a subsequent clause, breaking optimizer context tracking.

Bug-Exposing LPG:

```
CREATE (:L0 {k1: 10})-[:R1]->(:L1 {k2: 20})
```

Trigger Query:

```
MATCH (k:L0), (t:L1) WITH k, t
MATCH (k)-[:R1]->(t) WHERE k.k1 < t.k2
RETURN k.k1, t.k2
```

Root Cause:

- Optimizer fails to track variable scope transition across WITH boundaries
- Cross-variable dependency ($k.k1 < t.k2$) in single clause triggers incorrect plan rewriting

Bug Fix Findings 10 & 11: Fix Strategies & Quality

Finding 10: 30.5% of optimization bugs have at least one buggy patch.

- *Source:* 25/82 bugs required multiple fix iterations (higher than semantic/memory bugs).
- *Value:* Warns developers to use change impact analysis to avoid regression during patching.

Bug Fix Findings 10 & 11: Fix Strategies & Quality

Finding 10: 30.5% of optimization bugs have at least one buggy patch.

- *Source:* 25/82 bugs required multiple fix iterations (higher than semantic/memory bugs).
- *Value:* Warns developers to use change impact analysis to avoid regression during patching.

Finding 11: 32% of bugs are fixable via simple modifications.

- *Source:* 25/78 bugs fixed by tweaking conditions, parameters, or operators.
- *Value:* Strong promise for Automated Program Repair (APR) in database optimizers.

Bug Fix Findings 10 & 11: Fix Strategies & Quality

Finding 10: 30.5% of optimization bugs have at least one buggy patch.

- *Source:* 25/82 bugs required multiple fix iterations (higher than semantic/memory bugs).
- *Value:* Warns developers to use change impact analysis to avoid regression during patching.

Finding 11: 32% of bugs are fixable via simple modifications.

- *Source:* 25/78 bugs fixed by tweaking conditions, parameters, or operators.
- *Value:* Strong promise for Automated Program Repair (APR) in database optimizers.

Specific Example: RedisGraph-858

Fix Type: Simple condition change.

Detail: Adjusted a single boundary check in the rewrite rule, avoiding complex algorithm refactoring.

Tool Framework & Key Features

Architecture

LPG Generator → Cypher Query Generator → Oracle Checker → Bug Reporter

Tool Framework & Key Features

Architecture

LPG Generator → Cypher Query Generator → Oracle Checker → Bug Reporter

Key Features (Guided by Findings):

- **Lightweight Input (F6):** Generates simple/empty LPGs to maximize execution throughput.
- **Biased Query Generation (F7, F8):** CFG expansion prioritizes subqueries (20%) and variable reuse for high dependency complexity.
- **Clause Impact Analysis:** Tracks variable scope across subqueries to ensure semantic validity.
- **Cost-Effective Oracle (F9):** Focuses on internal errors, crashes, and version-diffed wrong results.

Core Workflow & Implementation

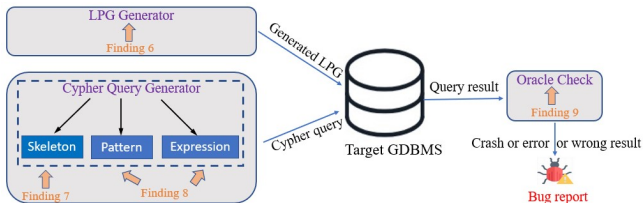


Figure: An overview of the architecture of the tool

Core Workflow & Implementation

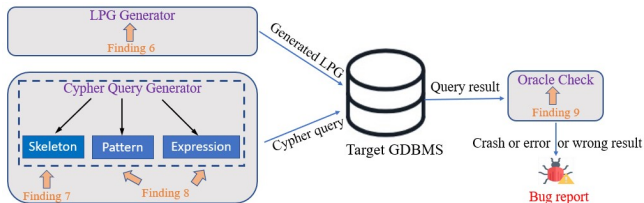


Figure: An overview of the architecture of the tool

- 1 **LPG Generation:** Fixed 128 nodes, 0–640 edges, 1–12 schema elements. Random but bounded.

Core Workflow & Implementation

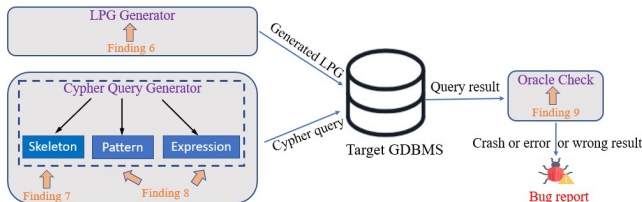


Figure: An overview of the architecture of the tool

- ❶ **LPG Generation:** Fixed 128 nodes, 0–640 edges, 1–12 schema elements. Random but bounded.
- ❷ **Query Generation:**
 - Builds skeleton via context-free grammar with biased probabilities.
 - Fills patterns/expressions by reusing previously bound variables.

Core Workflow & Implementation

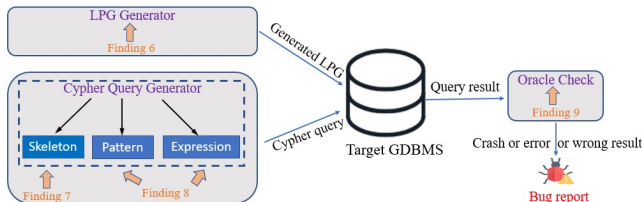


Figure: An overview of the architecture of the tool

- ❶ **LPG Generation:** Fixed 128 nodes, 0–640 edges, 1–12 schema elements. Random but bounded.
- ❷ **Query Generation:**
 - Builds skeleton via context-free grammar with biased probabilities.
 - Fills patterns/expressions by reusing previously bound variables.
- ❸ **Execution & Oracle:**
 - Runs queries on Neo4j/Memgraph.
 - Parses logs for **Internal Error/Crash**.
 - Diffs outputs against stable historical versions.

Typical Discovered Bugs

Case 1: IDP Exploration Defect (Neo4j)

Trigger: `shortestPath` alias referenced only inside `EXISTS{}`.

Bug: Planner fails to incorporate the alias into dependency modeling, treating it as unsolved.

Impact: Abnormal plan structure, re-planning overhead.

Typical Discovered Bugs

Case 1: IDP Exploration Defect (Neo4j)

Trigger: shortestPath alias referenced only inside EXISTS{ }.

Bug: Planner fails to incorporate the alias into dependency modeling, treating it as unsolved.

Impact: Abnormal plan structure, re-planning overhead.

Case 2: Graph State Mismanagement (Neo4j)

Trigger: Property caching optimization with variable renaming.

Bug: Cached attribute remains bound to original variable after scope change.

Impact: Runtime layout rebuild fails, throws Internal Error.

Note: Both cases triggered on empty LPGs, validating F6 and the tool's efficiency.

Comparison & Coverage

- **Missed Bugs:** Existing tools (GDSmith, GRev, GQS, etc.) missed long-latent optimization bugs present since 2022.
- **Coverage:** Our tool achieves higher line/branch coverage by exercising deep optimization paths (subqueries, complex dependencies).
- **False Alarms:** Lowest false alarm rate among compared techniques; focuses on deterministic failures.

Compared Techniques	Coverage		Bug Detection	
	Line	Branch	# Bug Reports	# Unique Bugs
GDSmith	18%	10%	465 (453)	1
GraphGenie	19%	10%	3 (3)	0
GAMERA	17%	9%	947 (837)	1
GRev	19%	11%	12 (12)	0
GraspDB	22%	13%	370 (370)	0
GSlicer	18%	10%	4 (4)	0
GQS	19%	11%	48 (46)	1
Our Tool	21%	13%	79 (2)	5

Figure: Tool outperforms baselines in unique bug detection and false alarm rate.

Summary & Contributions

- **First Empirical Study:** Large-scale analysis of 102 real-world GDBMS optimization bugs across 4 systems.
- **11 Novel Findings:** Systematically mapped root causes, manifestation patterns, and fix strategies.
- **Finding-Guided Tool:** Developed a customized tester that successfully exposed 20 unique bugs (10 optimization-focused).
- **Practical Guidance:** Provides actionable insights for optimizer developers (state tracking, formal rules) and APR researchers.

Open Source: <https://github.com/HumblePLSE/GDBQueryOptimization>